

Épreuve d'Algorithmes et Programmation Structurée (2h)

Exercice 1 : Manipulation de pointeurs (2,5pts)

Écrivez un algorithme qui déclare deux variables entières a et b, initialise a à 10 et b à 20. Ensuite, utilisez un pointeur pour :

1. Afficher la valeur de a.
2. Modifier la valeur de a à 30 via le pointeur.
3. Afficher la nouvelle valeur de a.
4. Échanger les valeurs de a et b en utilisant uniquement des pointeurs pour accéder et modifier leurs valeurs.
5. Afficher les valeurs finales de a et b.

Correction 1.1 :

```
Algorithme ManipulationPointeurs
Variables
  a, b : Entier
  ptrA, ptrB : Pointeur sur Entier
  temp : Entier

Début
  a ← 10
  b ← 20

  // 1. Initialiser les pointeurs
  ptrA ← &a
  ptrB ← &b

  // 2. Afficher la valeur de a via le pointeur
  Afficher "Valeur initiale de a (via pointeur) : ", *ptrA

  // 3. Modifier la valeur de a via le pointeur
  *ptrA ← 30
  Afficher "Nouvelle valeur de a (via pointeur) : ", *ptrA

  // 4. Échanger les valeurs de a et b via les pointeurs
  //   (Technique d'échange classique avec une variable temporaire)
  temp ← *ptrA
  *ptrA ← *ptrB
  *ptrB ← temp

  // 5. Afficher les valeurs finales
  Afficher "Valeur finale de a : ", a
  Afficher "Valeur finale de b : ", b
Fin
```

Exercice 2 : Insertion en tête dans une liste simplement chaînée (4pts)

Définissez la structure d'un nœud d'une liste simplement chaînée qui stocke un entier. Écrivez un algorithme pour insérer un nouvel entier en tête d'une liste chaînée existante. La fonction prendra en paramètre la tête de la liste (par référence, car elle pourrait changer) et la valeur à insérer.

Correction 2.1 :

```
Structure Noeud
    donnee : Entier
    suivant : Pointeur sur Noeud
Fin Structure

Algorithme InsertionEnTete
Paramètres
    TeteListe : Pointeur sur Pointeur sur Noeud (par référence, pour modifier la Tete)
    valeur : Entier

Variables
    nouveauNoeud : Pointeur sur Noeud

Début
    // 1. Allouer de la mémoire pour le nouveau nœud
    nouveauNoeud ← AllouerNoeud() // Fonction qui crée un nouveau Noeud et retourne son pointeur

    // 2. Assigner la valeur au nouveau nœud
    nouveauNoeud.donnee ← valeur

    // 3. Le nouveau nœud pointe vers l'ancien début de la liste
    nouveauNoeud.suivant ← *TeteListe

    // 4. Mettre à jour la tête de la liste pour qu'elle pointe vers le nouveau nœud
    *TeteListe ← nouveauNoeud

Fin

// Fonction principale pour tester
Algorithme TestInsertionListe
Variables
    maListe : Pointeur sur Noeud (initialisée à NIL/NULL)

Début
    maListe ← NIL // La liste est vide au départ

    Afficher "Insertion de 5..."
    Appeler InsertionEnTete(&maListe, 5) // &maListe passe l'adresse du pointeur maListe
    // La liste est maintenant : 5 -> NIL

    Afficher "Insertion de 10..."
    Appeler InsertionEnTete(&maListe, 10)
    // La liste est maintenant : 10 -> 5 -> NIL

    Afficher "Insertion de 2..."
    Appeler InsertionEnTete(&maListe, 2)
    // La liste est maintenant : 2 -> 10 -> 5 -> NIL

    // Pour vérifier, on pourrait implémenter une fonction d'affichage
    // Algorithme AfficherListe(Pointeur sur Noeud liste)
    //     tant que liste non NIL faire
    //         afficher liste.donnee
    //         liste ← liste.suivant
    //     fin tant que
    // Fin

Fin
```

Exercice 3 : Implémentation d'une pile avec une liste chaînée (4pts)

Utilisez la structure de nœud définie précédemment (Noeud avec donnée et suivant). Implémentez les algorithmes pour les opérations suivantes d'une pile :

1. InitialiserPile(Pile) : Initialise une pile vide.
2. EstPileVide(Pile) : Retourne vrai si la pile est vide, faux sinon.
3. Empiler(Pile, valeur) : Ajoute valeur au sommet de la pile.
4. Depiler(Pile) : Retire et retourne l'élément au sommet de la pile. Gérer le cas où la pile est vide.
5. SommetPile(Pile) : Retourne l'élément au sommet de la pile sans le retirer. Gérer le cas où la pile est vide.

Correction 3.1 :

```
// On réutilise la Structure Noeud définie précédemment
// Structure Noeud
//   donnee : Entier
//   suivant : Pointeur sur Noeud
// Fin Structure

// La Pile est représentée par un Pointeur sur Noeud (vers le sommet)
// type Pile = Pointeur sur Noeud

Algorithme InitialiserPile
Paramètres
    P : Pointeur sur Pointeur sur Noeud // Par référence pour initialiser P

Début
    *P ← NIL
Fin

Algorithme EstPileVide
Paramètres
    P : Pointeur sur Noeud // Par valeur, on ne modifie pas P

Retourne Booleen

Début
    Retourner (P == NIL)
Fin

Algorithme Empiler
Paramètres
    P : Pointeur sur Pointeur sur Noeud // Par référence pour modifier P (le
    sommet)
    valeur : Entier

Variables
    nouveauNoeud : Pointeur sur Noeud

Début
    nouveauNoeud ← AllouerNoeud()
    nouveauNoeud.donnee ← valeur
    nouveauNoeud.suivant ← *P // Le nouveau nœud pointe vers l'ancien sommet
    *P ← nouveauNoeud // Le nouveau nœud devient le sommet
Fin

Algorithme Depiler
Paramètres
```

```

    P : Pointeur sur Pointeur sur Noeud // Par référence pour modifier P (le
sommets)

Retourne Entier // La valeur dépilee

Variables
    valeurDepilee : Entier
    temp : Pointeur sur Noeud

Début
    Si EstPileVide(*P) Alors
        Afficher "Erreur : La pile est vide, impossible de dépiler."
        Retourner -1 // Ou lancer une exception, selon le langage
    Sinon
        temp ← *P                // Sauvegarder l'ancien sommet
        valeurDepilee ← temp.donnee
        *P ← temp.suivant        // Le nouveau sommet est le suivant de l'ancien
        LibérerNoeud(temp)       // Libérer la mémoire de l'ancien sommet
        Retourner valeurDepilee
    Fin Si
Fin

Algorithme SommetPile
Paramètres
    P : Pointeur sur Noeud // Par valeur, on ne modifie pas P

Retourne Entier // La valeur au sommet

Début
    Si EstPileVide(P) Alors
        Afficher "Erreur : La pile est vide, pas de sommet."
        Retourner -1 // Ou lancer une exception
    Sinon
        Retourner P.donnee
    Fin Si
Fin

// Exemple d'utilisation
Algorithme TestPile
Variables
    maPile : Pointeur sur Noeud

Début
    Appeler InitialiserPile(&maPile)

    Afficher "La pile est vide ? ", EstPileVide(maPile) // Vrai

    Appeler Empiler(&maPile, 10)
    Appeler Empiler(&maPile, 20)
    Appeler Empiler(&maPile, 30)
    // Pile: 30 (sommets) -> 20 -> 10 -> NIL

    Afficher "Sommet de la pile : ", SommetPile(maPile) // 30
    Afficher "La pile est vide ? ", EstPileVide(maPile) // Faux

    Afficher "Dépiler : ", Depiler(&maPile) // 30
    // Pile: 20 (sommets) -> 10 -> NIL

    Afficher "Sommet de la pile : ", SommetPile(maPile) // 20

    Afficher "Dépiler : ", Depiler(&maPile) // 20
    Afficher "Dépiler : ", Depiler(&maPile) // 10
    // Pile: NIL

    Afficher "La pile est vide ? ", EstPileVide(maPile) // Vrai
    Afficher "Dépiler : ", Depiler(&maPile) // Erreur, pile vide

Fin

```

Exercice 4 : Implémentation d'une file avec une liste chaînée (2,5pts)

Utilisez la structure de nœud (Noeud) définie précédemment. Définissez une structure File qui contient deux pointeurs : tete et queue. Implémentez les algorithmes pour les opérations suivantes d'une file :

1. InitialiserFile(F) : Initialise une file vide.
2. EstFileVide(F) : Retourne vrai si la file est vide, faux sinon.
3. Enfiler(F, valeur) : Ajoute valeur à la fin de la file.
4. Defiler(F) : Retire et retourne l'élément du début de la file. Gérer le cas où la file est vide.
5. TeteFile(F) : Retourne l'élément au début de la file sans le retirer. Gérer le cas où la file est vide.

Correction 4.1 :

```
// On réutilise la Structure Noeud
// Structure Noeud
//   donnee : Entier
//   suivant : Pointeur sur Noeud
// Fin Structure

Structure File
  tete : Pointeur sur Noeud
  queue : Pointeur sur Noeud
Fin Structure

Algorithme InitialiserFile
Paramètres
  F : Pointeur sur File // Par référence pour initialiser F

Début
  (*F).tete ← NIL
  (*F).queue ← NIL
Fin

Algorithme EstFileVide
Paramètres
  F : Pointeur sur File // Par valeur, on ne modifie pas F

Retourne Booleen

Début
  Retourner ((*F).tete == NIL) // Ou (*F).queue == NIL, les deux doivent
  être NIL si vide
Fin

Algorithme Enfiler
Paramètres
  F : Pointeur sur File // Par référence pour modifier F
  valeur : Entier

Variables
  nouveauNoeud : Pointeur sur Noeud

Début
  nouveauNoeud ← AllouerNoeud()
  nouveauNoeud.donnee ← valeur
  nouveauNoeud.suivant ← NIL // Un élément en fin de file pointe vers NIL
```

```

    Si EstFileVide(F) Alors
        // Si la file est vide, le nouveau nœud est à la fois la tête et la
queue
        (*F).tete ← nouveauNoeud
        (*F).queue ← nouveauNoeud
    Sinon
        // Sinon, le nœud actuel en queue doit pointer vers le nouveau nœud
        (*F).queue.suivant ← nouveauNoeud
        // Et le nouveau nœud devient la nouvelle queue
        (*F).queue ← nouveauNoeud
    Fin Si
Fin

Algorithme Defiler
Paramètres
    F : Pointeur sur File // Par référence pour modifier F

Retourne Entier // La valeur défilée

Variables
    valeurDefilee : Entier
    temp : Pointeur sur Noeud

Début
    Si EstFileVide(F) Alors
        Afficher "Erreur : La file est vide, impossible de défiler."
        Retourner -1 // Ou lancer une exception
    Sinon
        temp ← (*F).tete // Sauvegarder l'ancien début
        valeurDefilee ← temp.donnee
        (*F).tete ← temp.suivant // Le nouveau début est le suivant de l'ancien

        Si (*F).tete == NIL Alors
            // Si le défilement vide la file, la queue doit aussi être NIL
            (*F).queue ← NIL
        Fin Si

        LibérerNoeud(temp) // Libérer la mémoire de l'ancien début
        Retourner valeurDefilee
    Fin Si
End

Algorithme TeteFile
Paramètres
    F : Pointeur sur File // Par valeur

Retourne Entier // La valeur au début

Début
    Si EstFileVide(F) Alors
        Afficher "Erreur : La file est vide, pas de tête."
        Retourner -1 // Ou lancer une exception
    Sinon
        Retourner (*F).tete.donnee
    Fin Si
End

// Exemple d'utilisation
Algorithme TestFile
Variables
    maFile : File // Déclare une variable de type File

Début
    Appeler InitialiserFile(&maFile) // Passer l'adresse de maFile

    Afficher "La file est vide ? ", EstFileVide(&maFile) // Vrai

    Appeler Enfiler(&maFile, 10)
    Appeler Enfiler(&maFile, 20)

```

```

Appeler Enfiler(&maFile, 30)
// File: 10 (tête) -> 20 -> 30 (queue)

Afficher "Tête de la file : ", TeteFile(&maFile) // 10
Afficher "La file est vide ? ", EstFileVide(&maFile) // Faux

Afficher "Défiler : ", Defiler(&maFile) // 10
// File: 20 (tête) -> 30 (queue)

Afficher "Tête de la file : ", TeteFile(&maFile) // 20

Afficher "Défiler : ", Defiler(&maFile) // 20
Afficher "Défiler : ", Defiler(&maFile) // 30
// File: NIL

Afficher "La file est vide ? ", EstFileVide(&maFile) // Vrai
Afficher "Défiler : ", Defiler(&maFile) // Erreur, file vide

Fin

```

Exercice 5 : Insertion et recherche dans un ABR (4pts)

Définissez la structure d'un nœud d'un ABR qui stocke un entier. Chaque nœud aura des pointeurs vers son sous-arbre gauche et son sous-arbre droit. Implémentez les algorithmes pour les opérations suivantes dans un ABR :

1. RechercheABR(racine, valeur) : Retourne vrai si valeur est présente dans l'ABR, faux sinon.
2. InsertionABR(racine, valeur) : Insère valeur dans l'ABR en respectant les propriétés de l'ABR. La fonction doit modifier le pointeur racine si l'arbre est vide ou si la racine change (bien que pour l'insertion classique, seule la racine d'un arbre vide soit modifiée par référence).

Correction 5.1 :

```

Structure NoeudABR
    donnee : Entier
    gauche : Pointeur sur NoeudABR
    droit : Pointeur sur NoeudABR
Fin Structure

// La racine de l'ABR est un Pointeur sur NoeudABR
// type ABR = Pointeur sur NoeudABR

Algorithme RechercheABR
Paramètres
    racine : Pointeur sur NoeudABR // Par valeur, on ne modifie pas l'arbre
    valeur : Entier

Retourne Boolean

Début
    Si racine == NIL Alors
        Retourner Faux // Valeur non trouvée, arbre vide ou chemin terminé
    Fin Si

    Si valeur == racine.donnee Alors
        Retourner Vrai // Valeur trouvée
    Sinon Si valeur < racine.donnee Alors

```

```

        Retourner RechercheABR(racine.gauche, valeur) // Chercher dans le
sous-arbre gauche
    Sinon // valeur > racine.donnee
        Retourner RechercheABR(racine.droit, valeur) // Chercher dans le
sous-arbre droit
    Fin Si
Fin

Algorithme InsertionABR
Paramètres
    racine : Pointeur sur Pointeur sur NoeudABR // Par référence pour modifier
la racine si l'arbre est vide
    valeur : Entier

Variables
    nouveauNoeud : Pointeur sur NoeudABR

Début
    Si *racine == NIL Alors
        // L'emplacement d'insertion est trouvé (un nœud NIL)
        nouveauNoeud ← AllouerNoeudABR() // Fonction qui alloue et retourne
un Pointeur sur NoeudABR
        nouveauNoeud.donnee ← valeur
        nouveauNoeud.gauche ← NIL
        nouveauNoeud.droit ← NIL
        *racine ← nouveauNoeud // Le nouveau nœud est inséré ici
    Sinon Si valeur < (*racine).donnee Alors
        Appeler InsertionABR(&(*racine).gauche, valeur) // Insérer dans le
sous-arbre gauche
    Sinon Si valeur > (*racine).donnee Alors
        Appeler InsertionABR(&(*racine).droit, valeur) // Insérer dans le
sous-arbre droit
    Sinon
        // Valeur déjà présente (ou gestion des doublons, ici on ne fait rien)
        Afficher "La valeur ", valeur, " existe déjà dans l'ABR."
    Fin Si
Fin

// Exemple d'utilisation
Algorithme TestABR
Variables
    monABR : Pointeur sur NoeudABR

Début
    monABR ← NIL // L'ABR est vide au départ

    Afficher "Insertion de 50..."
    Appeler InsertionABR(&monABR, 50)
    // Arbre: 50

    Afficher "Insertion de 30..."
    Appeler InsertionABR(&monABR, 30)
    // Arbre: 50
    //      /
    //     30

    Afficher "Insertion de 70..."
    Appeler InsertionABR(&monABR, 70)
    // Arbre: 50
    //      / \
    //     30 70

    Afficher "Insertion de 20..."
    Appeler InsertionABR(&monABR, 20)
    // Arbre: 50
    //      / \
    //     30 70
    //      /
    //     20

```

```

Afficher "Insertion de 40..."
Appeler InsertionABR(&monABR, 40)
// Arbre: 50
//      /  \
//     30   70
//    /  \
//   20  40

Afficher "Recherche de 20 : ", RechercheABR(monABR, 20) // Vrai
Afficher "Recherche de 60 : ", RechercheABR(monABR, 60) // Faux
Afficher "Recherche de 50 : ", RechercheABR(monABR, 50) // Vrai
Afficher "Recherche de 99 : ", RechercheABR(monABR, 99) // Faux

```

Fin

Exercice 6 : Représentation d'un Graphe (3pts)

Contexte : Nous voulons modéliser un petit réseau de villes et de routes. Les villes sont A, B, C, D, E. Les routes sont les suivantes :

- A est reliée à B
- A est reliée à C
- B est reliée à D
- C est reliée à D
- D est reliée à E

Questions :

1. Dessinez ce graphe. Est-il orienté ou non orienté ?
2. Listez l'ensemble des sommets (V) et l'ensemble des arêtes (E).
3. Donnez le degré de chaque sommet.
4. Le graphe est-il connexe ? Justifiez.
5. Donnez un exemple de chemin de A à E.
6. Ce graphe contient-il un cycle ? Si oui, lequel ?

Corrigé Exercice 1 :

1. Dessin du graphe :

```

2.      A  --  C
3.     /      \
4.    B  --  D  --  E

```

Ce graphe est **non orienté** car les routes n'ont pas de sens unique spécifié. Si A est reliée à B, B est aussi reliée à A.

5. Ensemble des sommets (V) et arêtes (E) :

- $V = \{A, B, C, D, E\}$
- $E = \{A, B, A, C, B, D, C, D, D, E\}$ (on utilise des accolades pour les paires non ordonnées d'arêtes)

6. Degré de chaque sommet :

- $\deg(A) = 2$ (relié à B et C)
- $\deg(B) = 2$ (relié à A et D)
- $\deg(C) = 2$ (relié à A et D)
- $\deg(D) = 3$ (relié à B, C et E)
- $\deg(E) = 1$ (relié à D)

7. Le graphe est-il connexe ? Oui, le graphe est **connexe**. Pour toute paire de sommets, il existe un chemin qui les relie. Par exemple, de A à E, on peut passer par A-C-D-E.

8. Exemple de chemin de A à E :

- $A \rightarrow C \rightarrow D \rightarrow E$
- Autre exemple : $A \rightarrow B \rightarrow D \rightarrow E$

9. Ce graphe contient-il un cycle ? Oui, ce graphe contient un cycle. Un exemple est le cycle $A \rightarrow B \rightarrow D \rightarrow C \rightarrow A$.